

Backend Engineer Journey – Day 3 (Java Methods & Pass-by-Value)

1. Java is Always Pass-by-Value

Golden Rule:

Java is **always Pass-by-Value**.

Java kabhi object ya reference pass nahi karti.

Java **value ki copy** pass karti hai.

- Primitive types → Primitive value ki copy
- Objects → Reference ki value ki copy

2. Method Call

Jab bhi koi method call hota hai, Java Stack me ek **naya Stack Frame** create karti hai.

Example:

```
greet();
```

Flow:

```
main() Stack Frame
    ↓
greet() Stack Frame
```

Method complete hone ke baad us Stack Frame ko automatically remove kar diya jata hai.

3. Primitive Type Passing

Example:

```
int a = 10;

change(a);

void change(int x){
    x = 100;
}
```

Flow:

```
a = 10
↓
Copy
↓
x = 10
↓
x = 100
```

Original variable `a` change nahi hota.

Output:

10

4. Object Passing

Example:

```
Student s = new Student();

change(s);
```

Method:

```
void change(Student x){  
    x.name = "Sameer";  
}
```

Memory:

```
Main Stack  
  
s -----> Student Object  
  
Method Stack  
  
x -----> Same Student Object
```

Dono references same object ko point karte hain.

Object modify hoga.

Output:

Sameer

5. Reference Reassignment

Example:

```
void change(Student x){  
  
    x = new Student();  
  
    x.name = "Sameer";  
  
}
```

Yahan:

- Naya object create hota hai.
- Sirf local reference **x** change hota hai.
- Original reference **s** change nahi hota.

Output:

Ali

6. Difference

Object Modification

```
x.name = "Sameer";
```

✓ Same object modify hota hai.

✓ Sab references ko change dikhta hai.

Reference Reassignment

```
x = new Student();
```

✓ Sirf local reference change hota hai.

✓ Original caller unaffected rehta hai.

7. Stack Frame

Har method ke liye Stack me ek naya frame banta hai.

Stack Frame stores:

- Method Parameters
- Local Variables
- Primitive Values
- Reference Variables

Method complete hote hi Stack Frame remove ho jata hai.

8. Spring Boot Connection

Jab hum likhte hain:

```
userService.save(user);
```

To Spring object pass nahi karta.

Reference ki value pass hoti hai.

Ye Java ke Pass-by-Value rule ko hi follow karta hai.

Interview Takeaways

- Java is always Pass-by-Value.
 - Primitive variables ki value copy hoti hai.
 - Object pass nahi hota.
 - Reference ki value copy hoti hai.
 - Object modify karne se caller ko change dikhta hai.
 - Reference ko new object assign karne se caller ko koi effect nahi hota.
 - Har method call ke liye Stack Frame create hota hai.
-

Golden Lines (Remember Forever)

- Java is always Pass-by-Value.
 - Objects are never passed.
 - References are never passed.
 - Java passes a copy of the reference value.
 - Modifying an object is different from changing a reference.
 - Every method call creates a new Stack Frame.
 - Stack Frames are automatically removed after method execution.
-

Common Interview Trap

✗ Java is Pass-by-Reference.

✓ Incorrect.

✓ Correct Answer:

Java is always Pass-by-Value. For objects, Java passes a copy of the reference value.

Day 3 Summary

By the end of Day 3, you should be able to explain:

- How method calls work.
- What a Stack Frame is.
- Why Java is Pass-by-Value.
- Difference between object modification and reference reassignment.
- How this concept is used internally by Spring Boot.